

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science
and Commerce Department of Information Technology
(B.Sc.I.T Semester V)

AI

Practical – VIII

Roll No.:T010	Name: Sagar Kandari
Class: TYIT	Batch:1
Date of Assignment:1-08-2026	Date/Time of Submission:1-08-2026

Implement a decision tree classifier using the iris or any relevant dataset

Code:-

Iris dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

```
# Load dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
```

```
# Create model
```

```
model = DecisionTreeClassifier()
```

```
# Train model
```

```
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
```

Output:-

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
```

Netflix dataset

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Load the dataset
data = pd.read_csv("NetFlix - NetFlix.csv")

# Handle missing values
data = data.fillna("Unknown")

# Convert categorical columns into numeric values
encoder = LabelEncoder()

for column in data.columns:
    data[column] = encoder.fit_transform(data[column].astype(str))

# Features (all columns except 'type')
X = data.drop("type", axis=1)

# Target column
y = data["type"]
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Prediction
prediction = model.predict([X_test.iloc[0]])

print("Predicted Class:", prediction[0])
print("Actual Class:", y_test.iloc[0])
Output:-
```

```
Training Accuracy: 100.0
Testing Accuracy: 98.84467265725289
Predicted Class: 0
Actual Class: 0
```

Implement k fold cross validation on a classified model

Iris dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
iris = load_iris()

# Store data and target
X = iris.data
y = iris.target

# Create classification model
model = KNeighborsClassifier()

# Apply 5-fold cross-validation
scores = cross_val_score(model, X, y, cv=5)

print("Accuracy scores for each fold:")
print(scores)

print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
```

Output:-

```
Accuracy scores for each fold:
[0.96666667 1.          0.93333333 0.96666667 1.          ]
Average Accuracy: 97.33 %
```

Netflix dataset

```
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder

# Load Netflix dataset
data = pd.read_csv("/content/NetFlix - NetFlix.csv")

# Display column names
print(data.columns)
```

```
# Remove missing values
data = data.dropna(subset=["release_year", "type"])

# Feature
X = data[["release_year"]]

# Target
y = data["type"]

# Convert Movie and TV Show into numbers
encoder = LabelEncoder()
y = encoder.fit_transform(y)

# Create classification model
model = KNeighborsClassifier()

# Apply 5-fold cross-validation
scores = cross_val_score(model, X, y, cv=5)

print("Accuracy scores for each fold:")
print(scores)

print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
```

Output:-

```
Accuracy scores for each fold:
[0.51412067 0.61489089 0.64868337 0.68464997 0.6480411 ]
Average Accuracy: 62.21 %
```